

# LifeLoad: Life Management Simulator

## Java Micro Project Report

**Submitted By:** [Student Name]  
**Programme:** MCA — Semester 2  
**Subject:** Operating Systems Lab  
**Date:** March 2026

### 1. Introduction

#### 1.1 Overview

**LifeLoad: Life Management Simulator** is a full-stack, turn-based Java desktop application that simulates the journey of adulthood. A player starts at **Age 18** with limited funds and basic stats, then navigates the complexities of real life — managing careers, finances, health, relationships, and stress — week by week until they either **retire successfully** or suffer a **critical life failure**.

#### 1.2 Purpose & Problem Solved

Many young adults lack experience in managing finances, time, and work-life balance. LifeLoad solves this by **gamifying real-world decision making** in a risk-free simulation environment. Players learn:

- The consequences of financial mismanagement (debt, stress spikes).
- The value of continuous learning and skill progression.
- The impact of lifestyle choices (gym, socializing, rest) on long-term health.
- How market cycles (Boom, Recession, Crash) affect investments.

#### 1.3 Application Type

A **client-server Java application** built with a Spring Boot REST API backend and a JavaFX desktop frontend, connected over HTTP with JWT-secured authentication.

### 2. Modules / Functionalities

The application is structured into **six primary modules**, each with a well-defined responsibility.

| # | Module                   | Description                                                                                                                                             |
|---|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Authentication & Session | Secure JWT-based login and registration. Manages user sessions persistently.                                                                            |
| 2 | Core Game Engine         | Turn-based weekly lifecycle. Player performs actions (Work, Study, Rest, etc.) that consume energy and modify stats.                                    |
| 3 | Economy & Market         | Dynamic stock market with 4 states: BOOM, GROWTH, RECESSION, CRASH. Players invest in Stocks, Real Estate, and Startups.                                |
| 4 | Mini-Game Suite          | 6 interactive real-time mini-games (Budget Quiz, Typing Hustle, Memory Matrix, Deep Work Focus, Crisis Management, Corporate Ladder) for bonus rewards. |
| 5 | Event Engine & Rivals    | Probabilistic random events triggered by stats. NPC rivals who grow wealth alongside the player.                                                        |
| 6 | Timeline & Leaderboard   | A scrollable life history of all major events. A global leaderboard ranking players by final wealth and balance score.                                  |

Key Game Actions Available

- 📁 **Go to Work** — Costs Energy; earns Money based on Knowledge level.
- 📖 **Read & Study** — Costs Money + Energy; gains Knowledge + Motivation.
- 🏋️ **Go to Gym** — Costs Money + Energy; gains Health, reduces Stress.
- 🍷 **Socialize** — Costs Money + Energy; improves Relationships + Happiness.
- 😴 **Rest** — Recovers Energy + Health; reduces Stress.
- 🧘 **Meditate** — Reduces Stress; improves Happiness + Motivation.
- 🌐 **Network** — Costs Money; improves Reputation + Relationships.
- 💻 **Freelance** — High pay; costs Energy + Stress. Requires Knowledge.

3. Technology Used

3.1 Backend (Server)

| Technology                  | Purpose                                       |
|-----------------------------|-----------------------------------------------|
| Java 17                     | Core programming language                     |
| Spring Boot 3               | REST API framework                            |
| Spring Security             | Authentication and authorization              |
| JSON Web Tokens (JWT)       | Stateless session management                  |
| Spring Data JPA / Hibernate | Object-relational mapping and database access |
| MySQL                       | Relational database for persistent game state |
| Maven                       | Project build and dependency management       |

3.2 Frontend (Client)

| Technology               | Purpose                                   |
|--------------------------|-------------------------------------------|
| JavaFX 17                | Desktop UI framework                      |
| JavaFX Timeline API      | Real-time animation engine for mini-games |
| Jackson (ObjectMapper)   | JSON serialization/deserialization        |
| java.net.http.HttpClient | HTTP communication with REST API          |

3.3 Development Tools

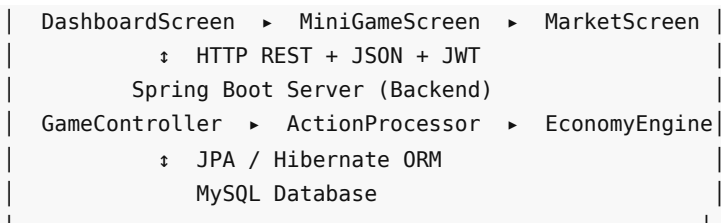
- **IDE:** IntelliJ IDEA / VS Code
- **Database Tool:** MySQL Workbench
- **Build Tool:** Apache Maven
- **Version Control:** Git

4. Implementation

4.1 System Architecture

The project follows a **3-Tier Client-Server architecture**:





## 4.2 Key Backend Classes

### GameController.java

The central REST controller. It handles all game-critical endpoints:

- `POST /api/game/start` — Initializes a new `PlayerProfile` and `PlayerStats`, seeds 3 NPC rivals, and records the first `TimelineEvent` ("START").
- `POST /api/game/action` — Delegates to `ActionProcessor`, then calls `advanceTime()` to tick a game week. Deducts living costs, ages the player, triggers market updates, and checks win/loss.
- `POST /api/game/minigame` — Applies minigame rewards/penalties directly to stats. Does **not** trigger `advanceTime()`, ensuring minigames are isolated from the weekly calendar.
- `GET /api/game/load` — Returns the full serialized game state (profile + stats) for session restoration.

### ActionProcessor.java

Encapsulates all action logic. For every action, it:

1. Validates the player has sufficient **Energy** and **Money**.
2. Applies trait-based multipliers (e.g., "Genius" trait boosts Knowledge gain by 30%).
3. Clamps all modified stats between `0` and `100`.
4. Returns a descriptive result message.

### EconomyEngine.java

Implements a **4-state financial market machine**:

- States: `BOOM` → `GROWTH` → `RECESSION` → `CRASH` → `BOOM`
- Each state has different growth/decay multipliers applied to all player investments when time advances.

### EventEngine.java

Probabilistic event system. Events fire based on:

- `Stress > 80` → may trigger "Burnout"
- `Happiness < 20` → may trigger "Depression"
- `Age >= 40` → aging health decay begins

### DatabaseSeeder.java

On startup, seeds the database with **Traits, Skills, Achievements**, and a large pool of **random life events** used by `EventEngine`.

## 4.3 Key Frontend Classes

### DashboardScreen.java

The main UI hub. Comprises three panels:

- **Left:** Live stat bars ( `ProgressBar` ) for Health, Energy, Happiness, Stress.
- **Center:** A `FlowPane` of action cards built dynamically from code. Low-money players see emergency actions (Odd Jobs, Desperate Hustle) appear.
- **Right:** NPC rival cards loaded asynchronously on a background thread.

### MiniGameScreen.java

Contains all 6 mini-game implementations using JavaFX's `Timeline` API for real-time animation. The result screen's "Collect Rewards" button fires a background thread sending data to `POST /api/game/minigame` .

**LifeLoadApp.java**

The entry point. On login success, calls `GameService.loadGame()` . If a save exists, navigates directly to `DashboardScreen` ; otherwise routes to `CharacterCreationScreen` .

**4.4 Security Flow**

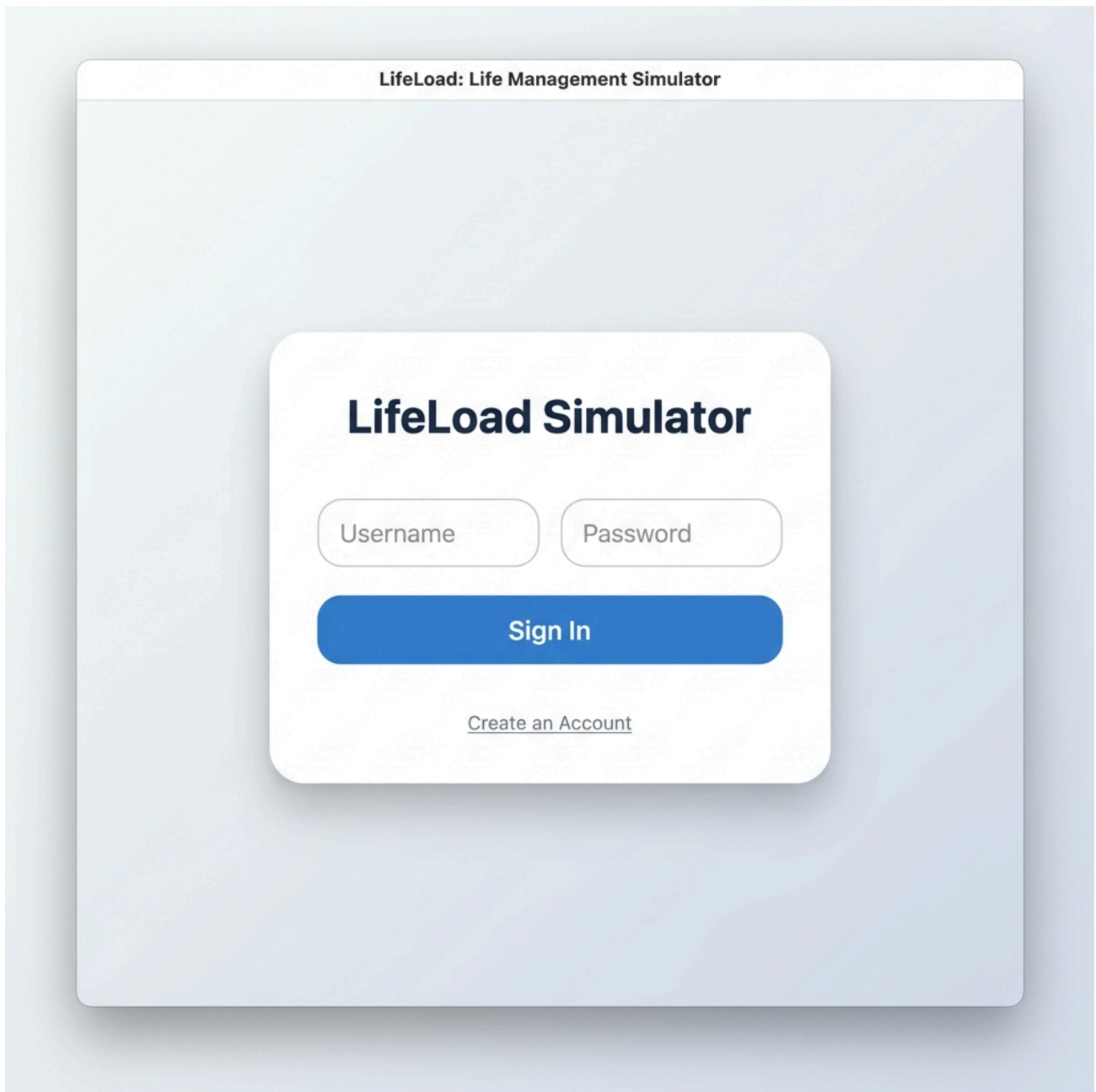
- 1. User logs in via `POST /api/auth/signin` .
- 2. Server validates credentials using `BCryptPasswordEncoder` .
- 3. On success, generates a **JWT token** signed with a secret key.
- 4. Client stores this token in `SessionManager` (in-memory).
- 5. All subsequent API calls attach `Authorization: Bearer <token>` in headers.
- 6. `AuthTokenFilter` intercepts every request, validates the JWT, and sets the `SecurityContext` .

**4.5 Game End Conditions ( `checkGameEnd` )**

| Condition                      | Outcome                                        |
|--------------------------------|------------------------------------------------|
| Health drops to 0              | <b>FAILED</b> — Died from health complications |
| Money < -\$5,000               | <b>FAILED</b> — Bankrupt                       |
| Age reaches 65                 | <b>WON</b> — Retired successfully              |
| Stress > 95 for multiple weeks | <b>FAILED</b> — Burnout                        |

**5. Screenshots with Explanation**

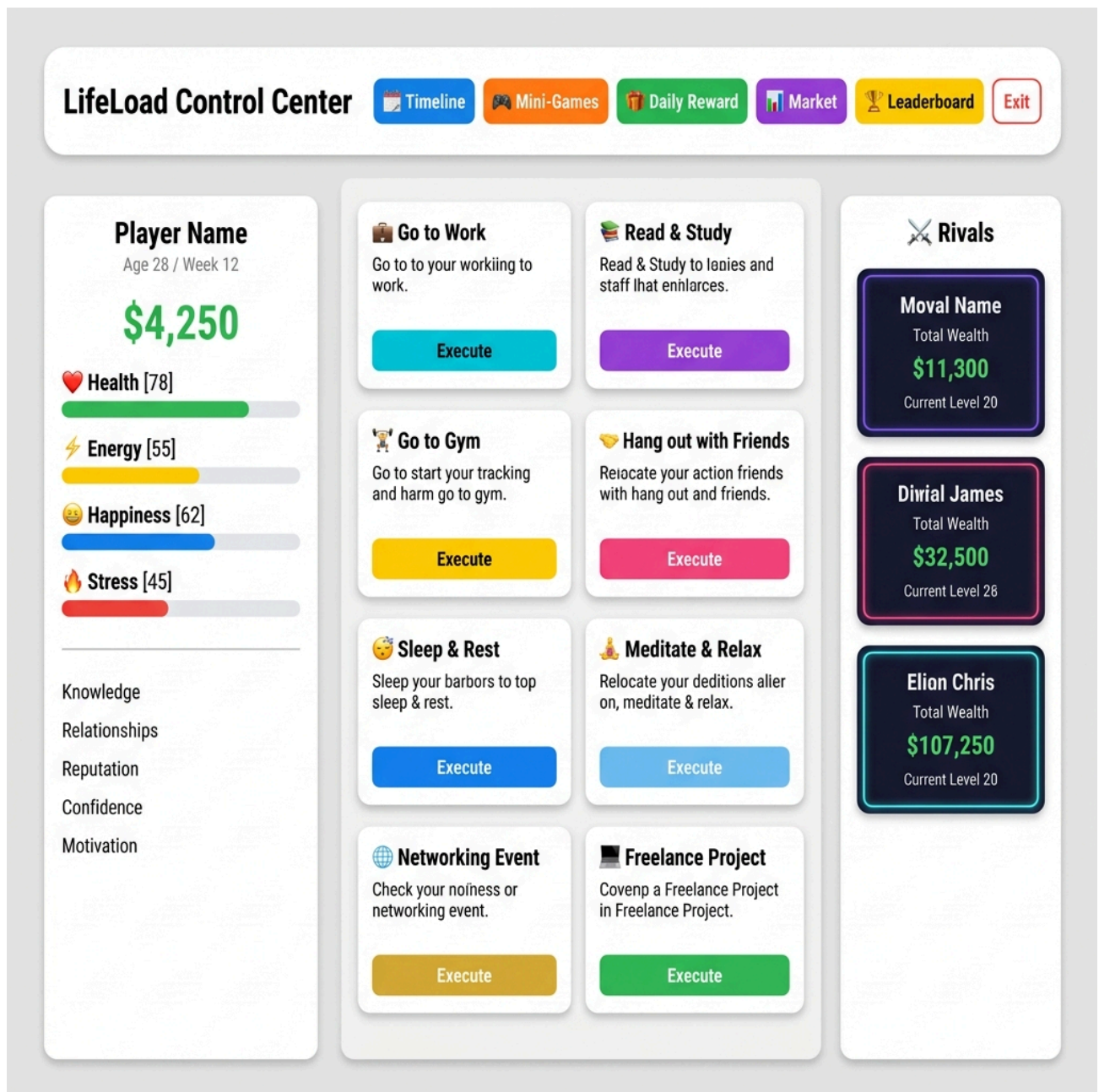
**Screenshot 1 — Login Screen**



**Explanation:** The secure entry point of the game. The white card-style UI sits on a soft gradient background. Players enter their username and password to sign in using the Spring Boot JWT authentication system. A "Create an Account" hyperlink navigates to the Registration screen. On successful login, the client calls `GET /api/game/load` to check for an existing save — if found, the player resumes from the Dashboard; if not, they proceed to Character Creation.

---

## Screenshot 2 — Main Dashboard (Control Center)



**Explanation:** The primary gameplay screen. The **top navigation bar** gives access to Timeline, Mini-Games, Daily Rewards, Market, and Leaderboard. The **left panel** displays real-time stat bars driven by live data from the backend — Health, Energy, Happiness, and Stress progress bars update after every action. The **center panel** shows 8 interactive action cards; each button fires `POST /api/game/action` to the server. The **right panel** dynamically lists NPC rivals loaded asynchronously via `GET /api/game/rivals`.

### Screenshot 3 — Global Economy Market Terminal



# GLOBAL ECONOMY MODULE

Current Market State: BOOM

Make an Investment:

STOCK ▼

Amount to Invest

EXECUTE TRADE

Your Portfolio:



STOCK Index

Bought: \$100,000

Value: \$1150,000

SELL



STOCK Index

Bought: \$40,000

Value: \$1330,00

SELL



STOCK Index

Bought: \$10,000

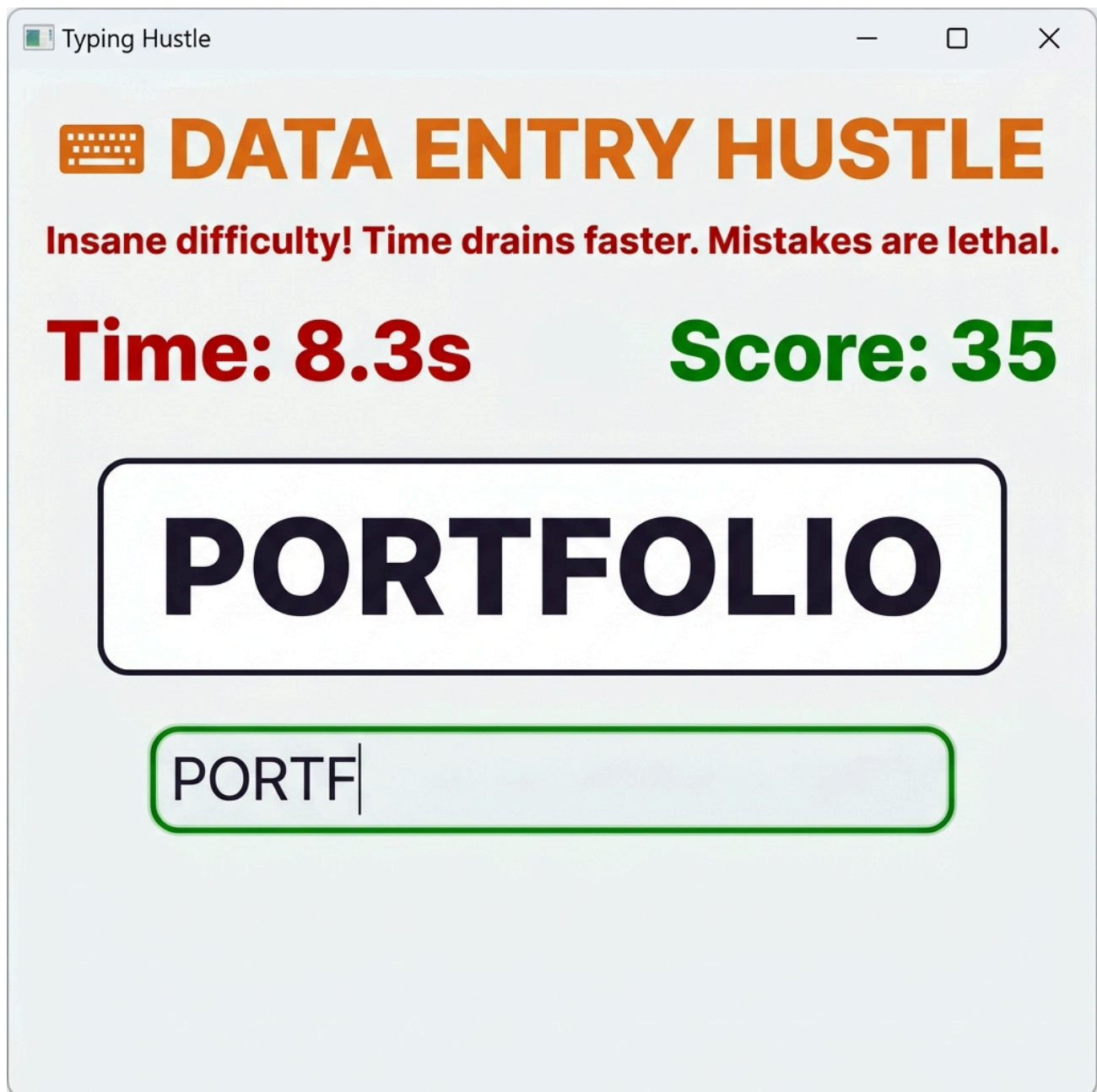
Value: \$-5,000

SELL

CLOSE TERMINAL

**Explanation:** The dark-themed investment terminal shows the live **Market State** (e.g., BOOM). Players select an investment type (STOCK, STARTUP, REAL\_ESTATE) and input an amount to execute a trade via `POST /api/economy/invest`. The portfolio table below shows all active investments with their initial purchase price vs. current market value — gains shown in green, losses in red. Each "SELL" button triggers `POST /api/economy/sell/{id}`, crediting the current market value to the player's balance.

**Screenshot 4 — Mini-Game: Typing Hustle (In Progress)**

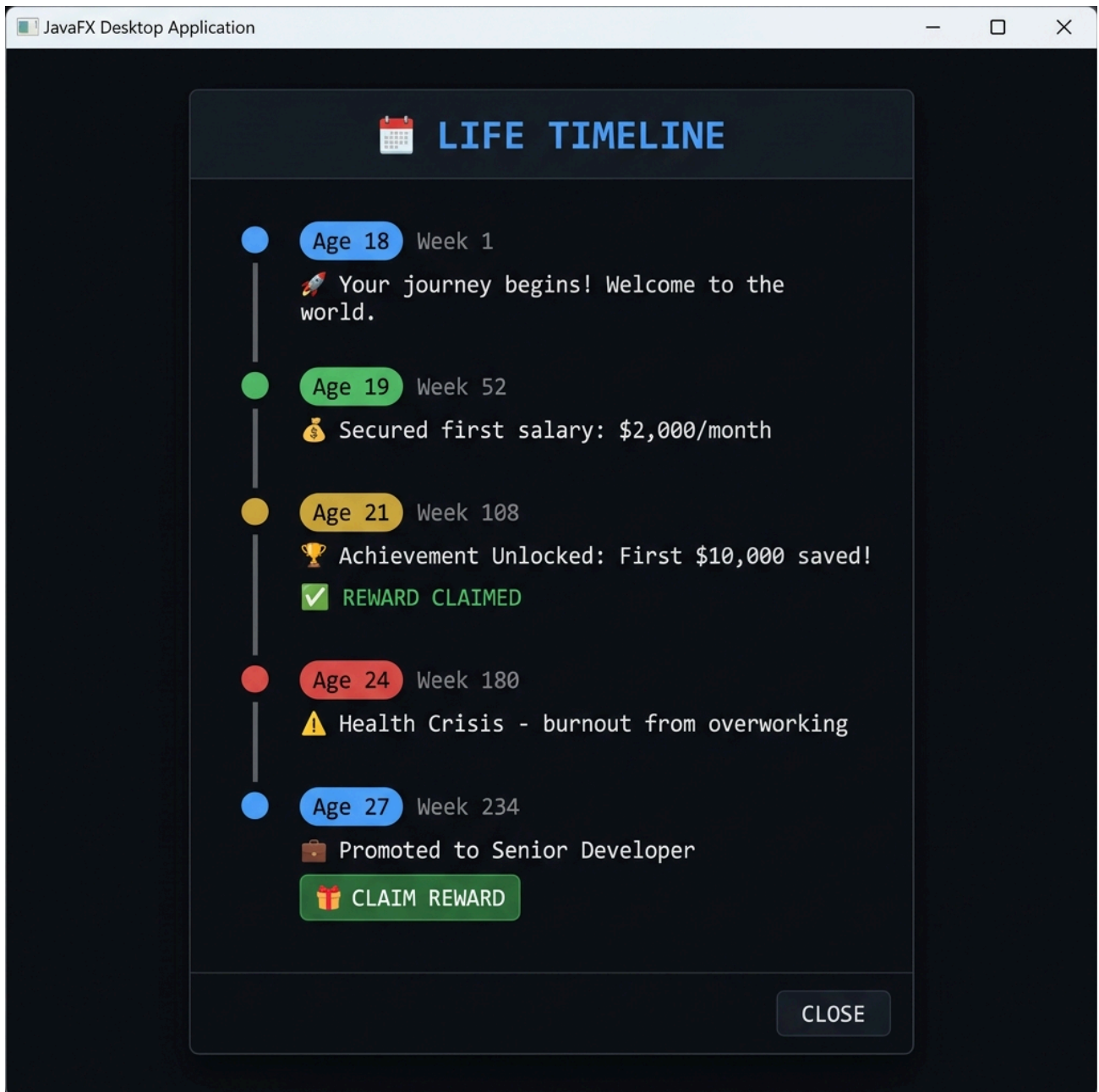


**Explanation:** One of 6 mini-games — a hardcore typing challenge. The **countdown timer drains faster** the higher the score, creating escalating pressure. The current target word is displayed in a large box; the player types it in the input field below (green border indicates correct prefix). A typo costs 2.5 seconds and \$20 penalty. This screen uses JavaFX's 16ms `Timeline` ticker for smooth real-time countdown. Scores are synced to the backend only when the game ends via `POST /api/game/minigame` .

---

#### Screenshot 5 — Life Timeline

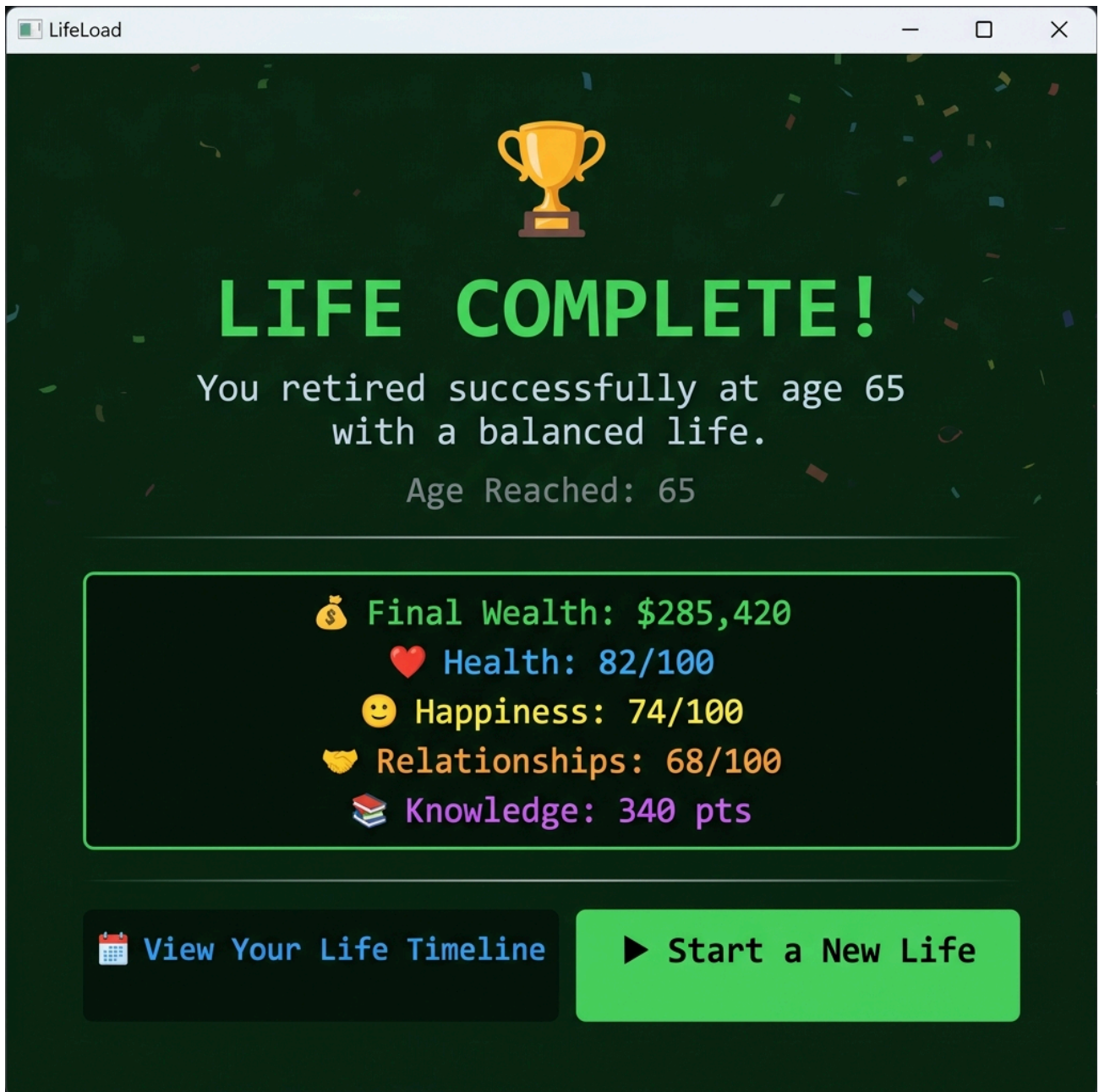




**Explanation:** A dark-themed chronological record of the player's life journey. Each entry shows the **age**, **week number**, and a description of the event. Color-coded dots indicate event category: ● Career, ● Money, ● Health Crisis, ● Achievement. Claimable milestone events show a "🎁 CLAIM REWARD" button that fires `POST /api/game/claim-milestone/{id}`. This data is fetched from the `timeline_events` database table via `GET /api/game/timeline`.

---

## Screenshot 6 — Game Completion Screen



**Explanation:** The final screen triggered when `checkGameEnd()` on the backend returns a terminal status. A **WIN state** (age 65 reached) shows "🏆 LIFE COMPLETE!" on a dark green background with a summary of final stats. A **LOSS state** shows "💀 GAME OVER" on a dark red background. The player can view their full Life Timeline or click "▶ Start a New Life" to go back to Character Creation.

## 6. Conclusion & References

### Conclusion

**LifeLoad: Life Management Simulator** successfully demonstrates a complete, production-quality full-stack Java application. The project achieves its core objective of gamifying life management and financial literacy through an engaging simulation engine.

Key achievements of the project:

- **Robust Architecture:** Clean separation of business logic (Spring Boot backend) from presentation (JavaFX frontend) via a REST API.

- **Security:** Stateless, token-based JWT authentication ensuring all game state is user-isolated.
  - **Engaging Gameplay:** A rich set of mechanics — stat clamping, market cycles, random events, mini-games, rival NPCs — provides genuine replay value.
  - **Educational Value:** Players learn core financial concepts (compound investment, market states, budget management) through gameplay rather than instruction.
  - **Technical Complexity:** Real-time JavaFX `Timeline` animations, multi-threaded network calls, and complex probabilistic event generation demonstrate advanced Java programming skills.
- 

## References

1. **Spring Boot Official Documentation**  
<https://spring.io/projects/spring-boot>
  2. **OpenJFX Official Documentation**  
<https://openjfx.io/openjfx-docs/>
  3. **Java 17 HttpClient API Reference**  
<https://docs.oracle.com/en/java/javase/17/docs/api/java.net.http/java/net/http/HttpClient.html>
  4. **JSON Web Tokens (JWT) Standard — RFC 7519**  
<https://jwt.io> / <https://datatracker.ietf.org/doc/html/rfc7519>
  5. **Spring Data JPA Reference Guide**  
<https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>
  6. **MySQL 8.0 Reference Manual**  
<https://dev.mysql.com/doc/refman/8.0/en/>
  7. **JavaFX Animation API (Timeline, KeyFrame)**  
<https://openjfx.io/javadoc/17/javafx.animation/javafx/animation/Timeline.html>
- 

*End of Report*